

Get It. — Technical Writeup

"What I cannot create, I do not understand." — Richard Feynman, last blackboard at Caltech, February 15, 1988.

Built at GDG AI Hack 2026, Milan — for the Braynr challenge.

By Mattia Beltrami (Politecnico di Milano), Matteo Impieri (Politecnico di Milano), Filippo Difronzo (Politecnico di Milano), Luca Feggi (Università di Padova).

The hackathon submission lived at commit `277ec43`. Everything you see here is that same product wrapped in a desktop shell so a student who has never opened a terminal can install it, sign in once, and have the same experience the judges saw. The product hasn't moved; the delivery has.

What we built and why it exists

The student already has the PDF. They don't need another summary; they need a way to see the parts of the document that text alone refuses to explain, and they need a way to *prove to themselves* that they have understood — concept by concept, not page by page. Today that proof is missing: ratings on flashcards measure recall in the moment, mindmaps measure how much you drew, summaries measure how patient the AI was. None of these measure whether the student would survive a question they hadn't seen before.

Get It. is built around two convictions. First, that **the document stays at the center**: every visualization, every chat, every flashcard, every Feynman session is grounded in the source PDF — never in general world knowledge. Second, that **mastery has structure**: it is not a single number, it is four orthogonal signals (memory, comprehension, structure, application) that move at different speeds and respond to different evidence. Get the structure right and the rest of the product writes itself.

We adopted Alessandro de Concini's own paradox as our brief: the Feynman technique is the most powerful learning tool ever conceived, and it is so slow that almost no one uses it across an exam syllabus. Our answer is not to abolish the slowness — depth costs time — but to remove the friction that has nothing to do with thinking: the blank page, the missing diagram, the lack of a listener. Four tools (chat, flashcards, forced-choice quizzes, Feynman-with-a-curious-child) feed a single per-document journal. One evaluator agent reads that journal end-to-end and updates a knowledge graph that the student can read back as a map of where they actually are.

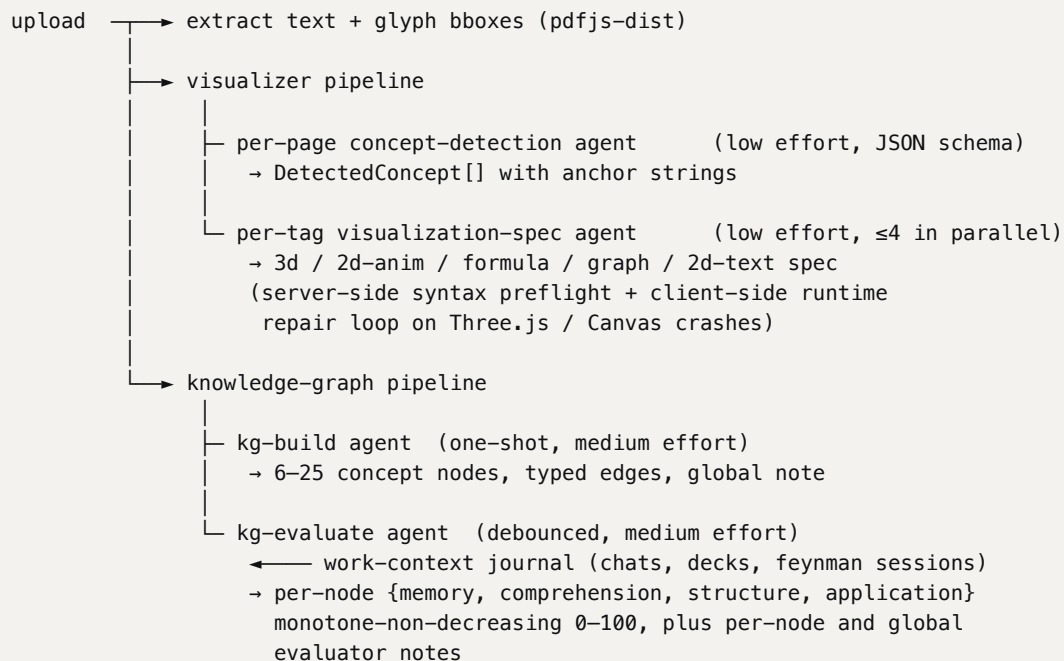
The macro user loop is therefore a closed cycle: the visualizer generates time-to-value the moment a PDF lands; the four tools generate study evidence; the evaluator turns evidence into a four-axis score per concept; the knowledge graph back-reflects that score onto the document and onto the next-best action. Output becomes outcome.

System architecture

The product is a single Next.js 16 application (App Router, React 19, server components for the entry points, client components for the orchestrators) wrapped in a small Electron shell so it ships as a desktop installer. There is no proprietary model in the loop — every agent call is a `codex exec` invocation through the official `@openai/codex-sdk`, constrained by a strict per-call JSON Schema so a "concept" or a "knowledge-graph node" is always a typed object, never free text the UI has to parse defensively.

The Electron shell does five things and gets out of the way: (1) on every launch — before the wizard, before any window — it queries GitHub Releases for a newer build of Get It. for the current platform/arch and, if one exists, surfaces a one-click installer flow that downloads the asset, opens it via the OS, and quits so the installer can replace the app; (2) on first launch it runs a setup wizard that verifies the bundled Codex CLI binary, prompts for the OAuth sign-in if needed, and refuses to open the main window until both gates are green; (3) it spawns the Next.js standalone server as a child Node process on a free localhost port and points a single Chromium `BrowserWindow` at it — the UI is exactly the browser experience, byte-for-byte; (4) it owns the per-user data directory (OS-native paths under `Application Support / %APPDATA% / ~/.local/share`) and exposes it to the Next side via `BRAYNR_DATA_DIR`; (5) it listens for Codex auth-loss or rate-limit signals and re-opens the wizard or surfaces a countdown banner without throwing away any work in flight.

Two macro-pipelines run **in parallel from the moment the PDF is uploaded**:



The visualizer pipeline is the one Braynr scores on time-to-value: tags appear inline in the document the instant detection returns; the right pane fills in by itself as visualizations land. Detection and per-tag generation are both **server-side singleton jobs**, one of each per docId, idempotent, running inside the Next process — not renderer loops. The detection job walks the unanalysed pages with concurrency 3 and persists each batch of new tags to `tags.json` as it goes; the viz queue picks the next tag with `generating: true`, runs the per-type agent with concurrency 4, persists the spec back to the same file. The viewer is a *consumer*: it polls `GET /api/tags/<docId>` every 1.5 s while any job is in flight, mirrors the response into local state, fires a `POST /api/jobs/viz/<docId>` on click or on sandbox runtime error. Navigating away, uploading another PDF, minimising the window — none of it stops the jobs. Multiple documents progress in parallel from any number of entry points; the library page polls the same source so its badges stay live across the whole catalog. When a visualization throws a runtime error in the sandbox we *do not* surface a stack trace — the viewer reports the message back to the server, which hands it to Codex as repair context (the broken `setup_code` plus the captured error) and the visualizer re-renders. The user sees "repairing — attempt N of M" instead of red text.

The knowledge-graph pipeline is the one that turns Get It. from a viewer into a measurement instrument. It is the layer the original Braynr architecture does not have, and it is where the four-axis rubric lives. Both agents talk to the same Codex provider as the visualizer — same SDK wrapper, same schema-enforced replies — so there is one auth path, one error mode, one budget.

The four-axis evaluator

Every concept node carries four scores from 0 to 100:

AXIS	WHAT IT MEASURES	STRONGEST SIGNAL
memory	recall over time	flashcard ratings (1–4), recall references in chat, FSRS-style stability
comprehension	understanding in the student's own words	original metaphors in chat, plain-language Feynman explanations
structure	grasp of how concepts connect	multi-step reasoning that bridges concepts, references to prerequisites
application	transfer to new cases	original examples, edge cases, novel problem solving

The evaluator agent sees the entire work context (compacted, with timestamps) plus the current graph plus the previous scores. It is told, in its system prompt, that scores are **monotone non-decreasing** — the student can only progress, never regress — and the runtime enforces this with a clamp on every update so a chatty interaction can't accidentally erase prior evidence. The agent is also told that *quantity does not entitle a score*: a student who has done fifty cards and rated them all "Again" sits low on memory; a student who has given a single brilliant Feynman

explanation can earn a strong jump on comprehension for that one concept. Empty work context produces no movement at all — the rubric is observable-evidence-based by construction.

Scheduling matters as much as the rubric. The evaluator is expensive (one Codex turn at medium effort per pass) and the chat tool is chatty by definition, so we run a per-doc queue with at most one in-flight pass and one pending. After every assistant chat reply, every ended flashcard deck, and every completed Feynman session, the API route fires `scheduleEvaluation(docId)` and returns immediately. The client polls `/api/kg/[docId]/state` (which exposes the live `evaluating` flag from the queue), accelerating to 2.5 s while the agent is working and slowing to 6 s when idle. The badge in the top tab bar reads "Building graph", "Evaluating", "No evaluations yet", or "Synced 12 s ago" depending on what the queue is doing.

The three study tools and the work-context journal

The four tools are deliberately small and deliberately different. **Chat** is the most familiar surface — multi-turn, multi-thread, scoped to one document, with the knowledge graph and a doc excerpt injected as system context.

Flashcards is the open-recall engine: Codex generates a 4–10 card deck for a topic, the student types their answer (optionally) and self-grades 1–4 (Again / Hard / Good / Easy, the FSRs convention) per card; ratings are recorded card-by-card and the deck is closed once every card is graded. **Quizzes** is the discrimination engine: Codex generates a 4–8 question multiple-choice quiz, each with one correct option and three plausible distractors crafted to expose the confusions a student would actually trip on; the student picks, gets immediate feedback with a one-sentence explanation, and the quiz ends with a score summary. Where flashcards measure recall under generation, quizzes measure recognition under temptation — a different signal the evaluator can lean on. **Feynman** is the showpiece: the agent plays a curious eight-year-old who asks three to four short, pointed questions; the student is forced into the role of the teacher; after the last turn a separate summary call writes a 3–6-sentence honest read of where the explanation held and where it broke down. The session is bounded so the data stays usable for the evaluator and the student doesn't drift.

Behind all four tools sits a single artifact: the **work-context JSON**, one file per doc on the server, append-only by convention. Every chat message, every card rating, every quiz answer, every Feynman turn lands here with a UTC timestamp. This is the file the evaluator reads, the file the student can download from the right-pane menu, and the file that — by design — is the only thing the system needs to remember about a study session. We give the student exactly what we use.

Implementation choices we are happy with

Persistent state is filesystem-backed under one OS-native data directory per user, resolved once in `lib/paths.ts` — `~/Library/Application Support/get-it/` on macOS, `%APPDATA%\get-it\` on Windows, `~/local/share/get-it/` on Linux (or whatever the Electron main process pinned via `BRAYNR_DATA_DIR`). Layout is `docs/<docId>/{source.pdf, extracted.json, meta.json, workctx.json, kg.json, tags.json}` with a top-level `docs.json` index so the Library renders the catalog without scanning the disk. Cheap, recoverable, OS-agnostic, and a clear seam if we ever lift it to a hosted backend. Tags + analysed-pages are now **server-owned** (written by the jobs runner, the route's POST handler only ever updates the active-tag selection) so a concurrent client navigation can never overwrite mid-flight detection or generation; the viewer reads via poll. Reopening a doc from the Library weeks later therefore restores the exact tag layout, viz specs and analysed-pages set without re-detection.

Types are split into `*-types.ts` modules (pure TS, no `node:fs` imports) and `*.ts` modules (the actual storage helpers). This sounds pedantic until you discover that Next.js bundles a transitively-imported module into the client when *any* type from it is referenced — even a bare `import type {}`. Splitting types into a node-free file is the only way to keep `lib/kg.ts` and `lib/work-context.ts` server-only without poisoning the browser bundle. We learned that the hard way and the comments in those files say so.

The visualizer sandbox runs LLM-emitted JavaScript inside a new `Function` IIFE with all dangerous globals (`window`, `document`, `fetch`, `XMLHttpRequest`, `WebSocket`, `Function`, `eval`, `localStorage`, `sessionStorage`, `require`) shadowed as `undefined` parameters. It is a defense against LLM mistakes, not a defense against adversarial input — the user runs their own Codex account against their own PDFs. The boundary is reasonable for the demo and explicitly documented.

Settings (auto-generate visualizations, max viz repair attempts) are runtime-mutable from a popover in the top tab bar. Both default to the env values from `lib/config.ts` (`NEXT_PUBLIC_AUTO_GENERATE_VIZ`, `NEXT_PUBLIC_MAX_VIZ_GEN_RETRIES`) and persist to `<DATA_DIR>/settings.json` — the dynamic localhost port the packaged app binds to changes on every launch, so anything cookie- or localStorage-scoped to the origin would forget the user's choice; a plain file in the user-data dir is the only thing that survives a restart. A change broadcasts a `getit:settings` window event so other pages on the same renderer react without polling: the viewer's running orchestration picks up the new auto-generate state mid-document; the server-side detection job reads the same file when it queues new tags. Toggling auto-generate from off to on sweeps already-detected idle tags into the viz queue immediately; on to off lets in-flight calls finish naturally.

Resilience to Codex outages

Every Codex call funnels through one helper, `lib/codex.ts → runJson`. That helper classifies failures into four kinds — `auth_lost`, `rate_limit` (with the 5-hour / weekly window pulled out of the error message when present), `binary_missing`, and `generic` — and writes the latest one into a process-local **health mailbox**. Two things hang off the mailbox:

1. **The in-app banner.** The renderer polls `/api/codex/health` (fast cadence while there's an active problem, slow cadence otherwise) and renders `components/CodexHealthBanner.tsx`. On auth loss it offers a "Re-connect" button that re-opens the desktop setup wizard via the Electron preload bridge. On rate-limit it counts down to `retryAt` and disappears on its own when the deadline passes.
2. **The kg-evaluator queue.** Hitting a rate-limit inside an evaluator pass schedules a `setTimeout` for `retryAt + 500 ms` that re-fires `scheduleEvaluation(docId)` — so the graph keeps catching up on its own without the user having to do anything. The build agent does the same: it leaves the KG in `status: "building"` instead of erroring out, so the badge keeps spinning and the next attempt picks up cleanly. Tool routes (chat / flashcards / Feynman) preserve the work-context journal up to the failure point, so the user can re-send the same action once the banner clears and pick up exactly where they were — no lost messages, no orphan card ratings, no half-finished Feynman session.

The `runJson` helper also short-circuits Codex calls while a rate-limit window is still active, so a chatty UI can't burn a hundred wasted calls hoping the next one will succeed. This keeps the recovery clean: one failure, one banner, and the rest of the app keeps working on cached state.

Desktop packaging

The Electron shell is the boring kind of shell — it does as little as possible. `electron/main.js` acquires a single-instance lock, normalises the user-data directory to `get-it` (we override Electron's default `Application Support/Get It` so the OS-native path matches the pure-Next dev default), runs the setup wizard, spawns the Next.js standalone server as a child Node process on a free port, and points one Chromium window at `http://127.0.0.1:<port>`. There is no native menu reinvention, no custom IPC for application data, and no second renderer — the user-visible UI is the unchanged Next.js app. We chose Electron over Tauri specifically because we wanted a guaranteed Chromium runtime on all three operating systems: Three.js scenes, KaTeX-rendered formulas, the `new Function(...)` LLM sandbox, and pdf.js fonts all behave identically on every machine the user can install on.

`electron/setup.js` owns the Codex life-cycle. The Codex CLI binary ships *inside* the app — it's a Rust binary packaged as an npm optional dependency (`@openai/codex-<platform>-<arch>`) that the SDK locates via `createRequire`. At build time `scripts/electron-prepare.mjs` fetches the correct platform tarball from the npm registry (so a cross-arch build from an M-series Mac can still produce a usable Windows installer) and stages it under `electron/codex-bin/<triple>/codex/codex(.exe)`. At runtime the setup module resolves that path first; if for any reason it's missing or out of date, a "Install Codex CLI" button downloads it on demand into the user-data dir. The OAuth sign-in is run by spawning `codex login` and capturing the success line from stdout — the binary opens the browser itself; if it can't, the URL is also surfaced in the wizard window with an "Open in browser" button. The wizard is a stand-alone `BrowserWindow` loaded from a plain `file:///` page with its own minimal preload bridge.

Multi-target builds are driven from `scripts/build-electron.mjs`. Local: `node scripts/build-electron.mjs --target=mac-arm64 | mac-x64 | win-x64 | --all`. CI: a tagged push (`v*.*.*`) to `main`

triggers `.github/workflows/release.yml`, which first rewrites `package.json#version` from the pushed tag so the same number flows into `Info.plist / NSIS metadata`, into `NEXT_PUBLIC_APP_VERSION` for the in-app version chip, and into the asset filenames; then builds each target on its native runner (`macos-14`, `macos-13`, `windows-latest`), uploads the `.dmg` / `.exe`, and a final `publish` job collects everything and creates the GitHub Release tied to the same tag. Builds are unsigned by default; users dismiss the first-launch Gatekeeper / SmartScreen warning once and it's not seen again. Real signing certificates are a deliberate choice deferred to a sustainable funding moment, not a missing piece of the architecture.

Auto-update closes the loop. On boot, `electron/updater.js` calls the GitHub Releases API for `beltromatti/get-it`, picks the asset whose filename matches the running platform and arch, and semver-compares its tag against `app.getVersion()` (the value the CI step pinned). When a newer version is out, a polished `BrowserWindow` shows the release notes and one "Update now" button; clicking it downloads the asset with a live progress bar, hands the file to `shell.openPath` (Finder mounts the `.dmg`, Windows runs the NSIS installer, Linux surfaces the `.AppImage` to the file manager), and quits so the installer can replace the app on disk. The user's library, work-context journals, knowledge graphs and settings all live outside the app bundle, so an in-place install never touches them. Network failures, 404s when no release is published yet, and assets missing for the running platform all silently bypass — the rest of startup proceeds unaffected.

Bring-your-own-account as an architectural choice

The decision to drive every agent through the **user's own ChatGPT login over the official Codex CLI** is not a cost-cutting move — it is the architectural choice that shapes the entire product. There is no server-side OpenAI key, no shared pool of credits, and no app-side metering of model usage. The Electron shell bundles the Codex CLI binary per platform/arch, the first-launch wizard spawns `codex login` so the user authenticates against OpenAI directly, and every subsequent `codex exec` call runs against that account at whatever tier the user pays for. The app sees the same auth state Codex itself sees: a successful login, a rate-limit window, an expired token. Nothing more.

Three things follow from that:

1. **No second subscription, ever.** Every other AI-study tool we benchmarked layers a marked-up monthly fee on top of an API key the vendor holds — the student ends up paying twice for the same model access. Get It. cannot do that because we never hold the key in the first place. ChatGPT Plus is the practical floor for sustained study sessions (the free tier signs in but its Codex allowance is intentionally small), and everything above Plus — Pro, Team, Enterprise, Edu — just gives the same flow more headroom, no app-side switch. This is the single sentence that explains the product positioning: *one subscription, the one you already have, no markup.*
2. **No data resale and no transit-stage intermediary.** Because we never proxy the model traffic through our infrastructure, there is no Get It. infrastructure for that traffic to flow through. The Codex CLI talks to OpenAI on the user's machine; we observe nothing. The work-context journal, the knowledge graph, the per-doc folders all live under an OS-native user-data directory on local disk. There is no upload step, no opt-in cloud sync, no analytics SDK. We sometimes describe this as "downloadable in one click" because the same JSON file the evaluator reads is the one the user can pull out from the right-pane menu — but the more honest framing is that there is nothing else to download: this is the only state, and it is already on the user's computer.
3. **The transport is replaceable.** Codex CLI is one of several ways the app could speak to a model. Today it is the one we ship because it has by far the best ergonomics around per-tier login, the bundled binary is small, the official SDK gives us schema-typed responses without DIY JSON-Schema enforcement, and it is the only path through which a ChatGPT Plus account can drive a developer-facing CLI without an extra API-key purchase. If OpenAI deprecates Codex, or if a comparable bring-your-own-account transport for another provider emerges, the `runJson` helper in `lib/codex.ts` is the only place that has to change.

The same property protects the project legally. **Get It. is not affiliated with OpenAI, not endorsed by OpenAI, not sponsored by OpenAI**, and not a derivative work of any closed-source OpenAI software — it is an independent application that interoperates with the publicly released Codex CLI and uses the end user's own credentials. We are careful to say so explicitly: in the README's Notice section, in the in-app first-launch wizard, and again at the bottom of this document. The user's actual use of OpenAI's models through Get It. is governed by OpenAI's own Terms of Use, Usage Policies, and Privacy Policy — those documents are authoritative for what is and isn't allowed, what happens to the data on OpenAI's side, and what each subscription tier covers. We have no insight into that loop and no way to override it.

The flip side is honesty about what the user is paying when they upgrade their ChatGPT tier: they are paying OpenAI for model access; they are not paying us. This is the inverse of how almost every AI-wrapper product is priced. We treat it as a feature.

What's out of scope and why

A vocal mode for Feynman is the natural next step — the same agent loop, the same end-of-session summary, with a streaming TTS layer over the child voice. We built the text variant for the hackathon because typed transcripts are strictly better evaluator inputs (no transcription error, no per-token cost, full searchability) and because the role-reversal UX — *you* are teaching, *the agent* is the audience — is what changes the affect, not the modality. Audio is the obvious extension; the data shape doesn't change.

A multi-user backend, real auth, and a hosted deployment are also deliberately out of scope. Get It. runs locally against the user's own Codex login, against their own PDFs, on their own machine, by design. The Braynr policy band — *source-grounded only, local-first, tiered access* — is something we get for free at this scale; we did not have to architect away from it.

What we want a judge to remember

Four things. (1) The document is the center, and every other surface back-reflects to it — the visualizer pulls from the page text, the chat injects the page text, the evaluator reads the journal *and* the page text. (2) Mastery is four numbers and they only ever go up — that one constraint is the difference between a study app and a measurement instrument. (3) The same Codex provider drives every agent — concept detection, visualization generation, knowledge-graph build, evaluator, chat, flashcards, quizzes, Feynman child, Feynman summary — nine prompts behind one auth path, nine schemas behind one shared SDK wrapper. (4) A student who has never opened a terminal can use this: a 200-MB installer, a one-time browser sign-in, and the same product the judges saw. The system is a sum of small, schema-typed turns, not a god-prompt; that is what makes it debuggable and that is what will let it grow.

"Their knowledge is so fragile." — Feynman, 1985. We took the line literally.

Notice and license

Get It. is an independent project. It is not affiliated with, endorsed by, sponsored by, or otherwise associated with OpenAI. The app uses the official open-source [Codex CLI](#) as the transport between the local app and OpenAI's models, signed in with the end user's own ChatGPT or OpenAI API account. "OpenAI", "ChatGPT", and "Codex" are trademarks of their respective owner — we use the names only to describe what Get It. interoperates with.

The user's use of OpenAI's models through Get It. is subject to OpenAI's own [Terms of Use](#), [Usage Policies](#), and [Privacy Policy](#), and to the Codex CLI's [own license and release notes](#). Those documents are the authoritative source for what the underlying model service permits and how data is handled on OpenAI's side.

Source code for Get It. is licensed under the **Apache License, Version 2.0** — see [LICENSE](#) in the repository root.